

From Linear Models to Neural Networks

Linear & logistic regression, neurons, MLPs, universal approximation

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Lecture 1 chose the loss; Lecture 2 expands the prediction rule

Keep from Lecture 1

Regression output $\rightarrow$ Gaussian model
 $\rightarrow$ MSE

Class probabilities $\rightarrow$ Bernoulli /
categorical model $\rightarrow$ cross-entropy

Change in Lecture 2

Start with one weighted sum, expose what it cannot represent, then let hidden layers learn nonlinear features.

$x \rightarrow f_{\theta}(x) \rightarrow$ output parameters $\rightarrow$ same Lecture 1 loss

We will not re-derive the losses; today is about making f_{θ} more expressive.

One question drives the lecture: **what changes when one weighted sum becomes a network?**



weighted sum → expose the limitation → learn nonlinear features → compose them

We will calculate every stage for small numerical examples before writing the general matrix form.

One notation covers every prediction rule

Write $f_{\theta}(x)$ for the model's prediction at input x .

x : the input features

θ : all trainable weights and biases

f_{θ} : the rule that maps input to a score or prediction

Starting point — one weighted sum plus a bias:

$$f_{\theta}(x) = w^{\top} x + b$$

We will build the network gradually; the full nested formula comes only after each part is concrete.

The linear starting point

Start with one weighted sum

For an input vector $x \in \mathbb{R}^d$ and scalar target y ,

$$\hat{y} = w^\top x + b$$

For $x = (2, -1)$, $w = (1.5, 0.5)$, and $b = -0.2$:

$$\hat{y} = 1.5(2) + 0.5(-1) - 0.2 = 2.3.$$

Regression: read 2.3 as the predicted mean.

Classification: read the weighted sum as a score, then convert it to probabilities.

The same weighted-sum core can feed different output models and losses from Lecture 1.

A whole batch is one matrix multiplication

For $\mathbf{X} \in \mathbb{R}^{n \times d}$ and $\mathbf{y} \in \mathbb{R}^n$,

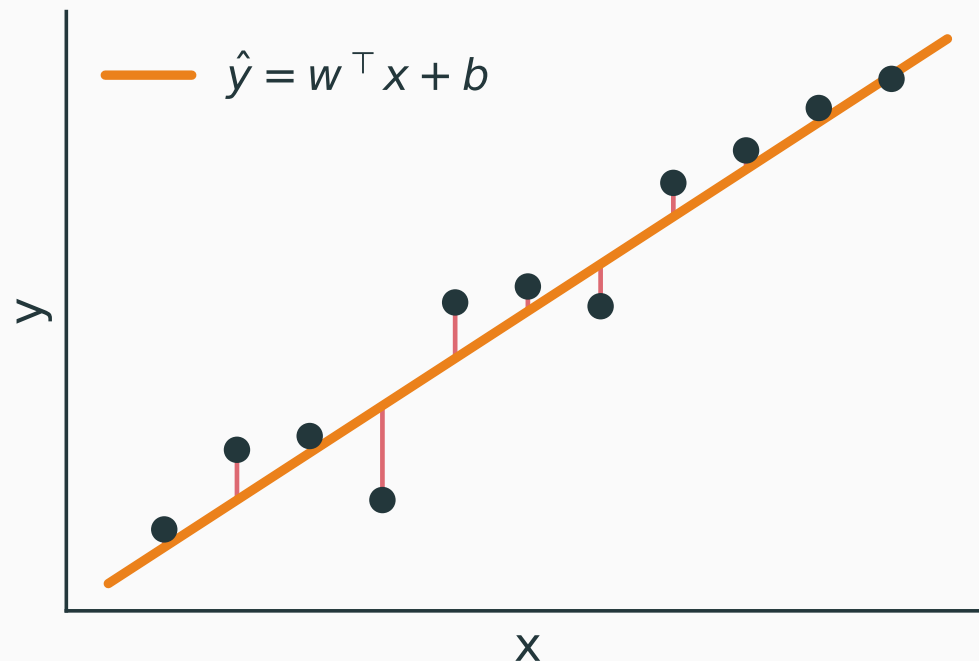
$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b\mathbf{1} = \tilde{\mathbf{X}}\boldsymbol{\theta}.$$

$$\tilde{\mathbf{X}} \in \mathbb{R}^{n \times (d+1)}, \quad \boldsymbol{\theta} \in \mathbb{R}^{d+1}, \quad \hat{\mathbf{y}} \in \mathbb{R}^n.$$

Rows are examples; columns are features; matrix multiplication performs all n predictions together.

For regression, one weighted sum gives one flat trend

$$\hat{y} = w^{\top} x + b$$

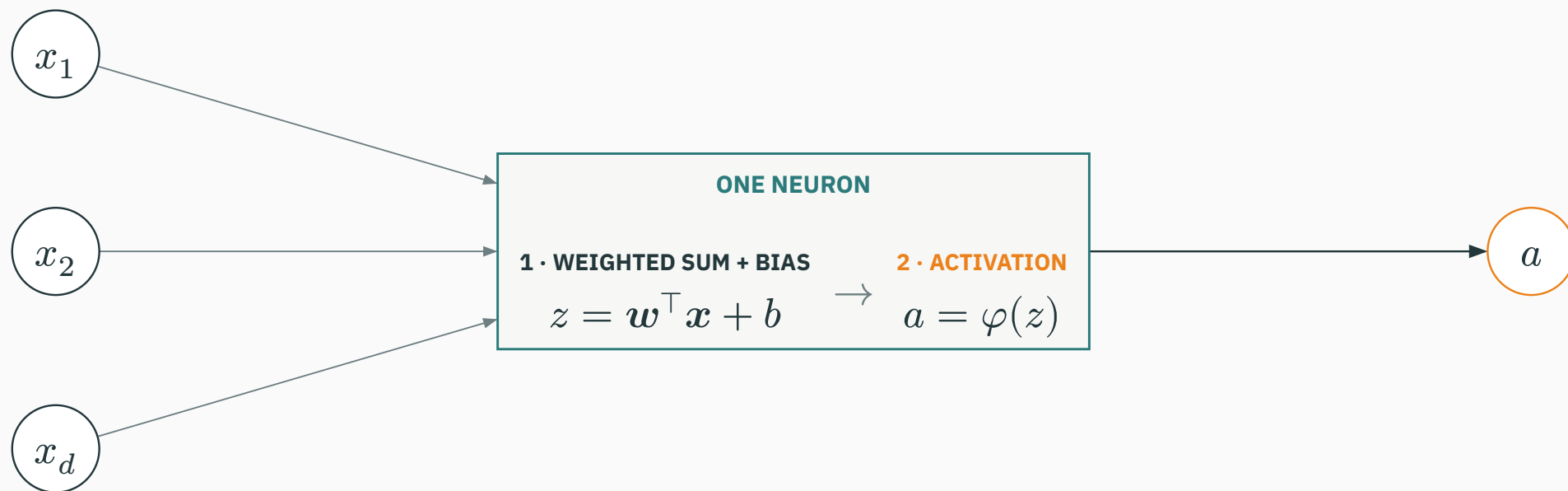


The model can tilt and shift a line or plane, but it cannot bend to follow nonlinear structure.

From a linear score to a neuron

A neuron has two stages

First compute a weighted sum plus bias; then pass that value through an activation.



One neuron recovers familiar Lecture 1 models

Let $z = \mathbf{w}^\top \mathbf{x} + b$. The output interpretation chooses the last step:

Regression

identity output

$$\hat{y} = z$$

Binary class

sigmoid output

$$p = \sigma(z)$$

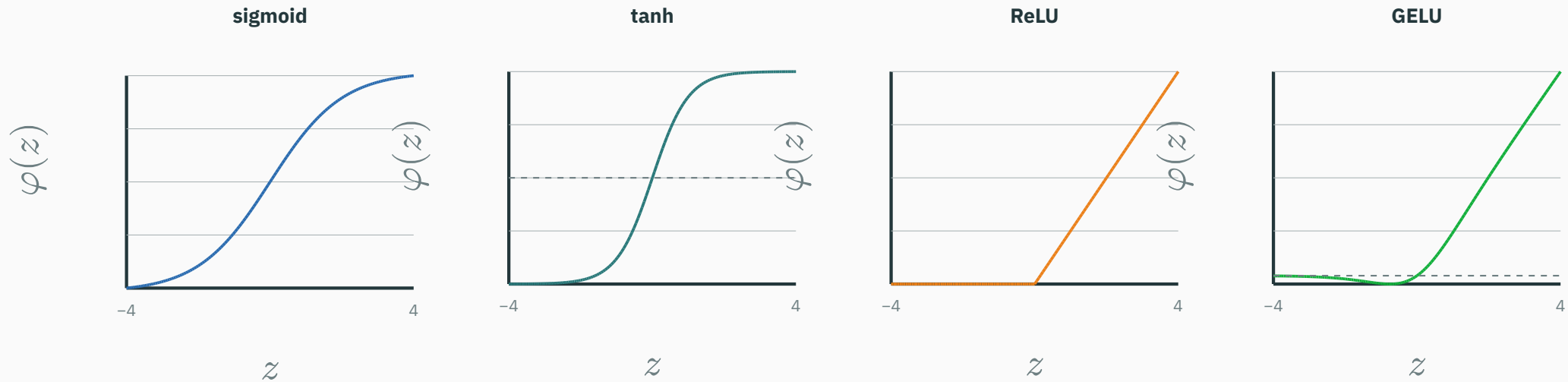
K classes

K scores + softmax

$$\mathbf{p} = \text{softmax}(\mathbf{z})$$

**Lecture 1 tells us how to interpret the output and choose the loss;
Lecture 2 changes the features entering that output.**

Common activation functions



sigmoid $\rightarrow$ gates/probabilities $\cdot$ tanh $\rightarrow$ zero-centred $\cdot$ **ReLU** $\rightarrow$ common hidden unit $\cdot$ GELU $\rightarrow$ Transformers

Remove every activation. What can a stack of “weighted sum + bias” layers represent?

A. One equivalent weighted sum + bias

B. Any curved decision boundary

C. Only a constant function

D. A probability distribution automatically

Algebraic view

$$h_1 = W_1 x + b_1$$

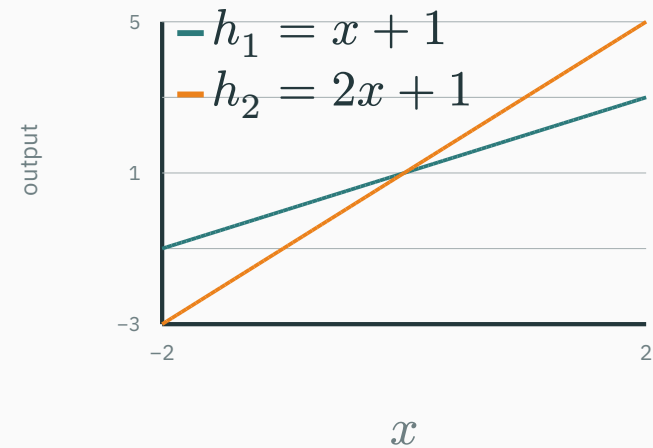
$$h_2 = W_2 h_1 + b_2$$

$$h_2 = \underbrace{W_2 W_1}_{\text{one matrix}} x + \underbrace{(W_2 b_1 + b_2)}_{\text{one bias}}$$

$W_2 W_1$ is one new weight matrix; the remaining term is one new bias vector.

Geometric view

$$h_1 = x + 1, \quad h_2 = 2h_1 - 1 = 2x + 1$$



The slope changes; no bend appears.

Correct: A. Without a nonlinear activation, extra layers still reduce to one weighted sum plus bias.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Compare activations and their **derivatives**, and see where gradients vanish.

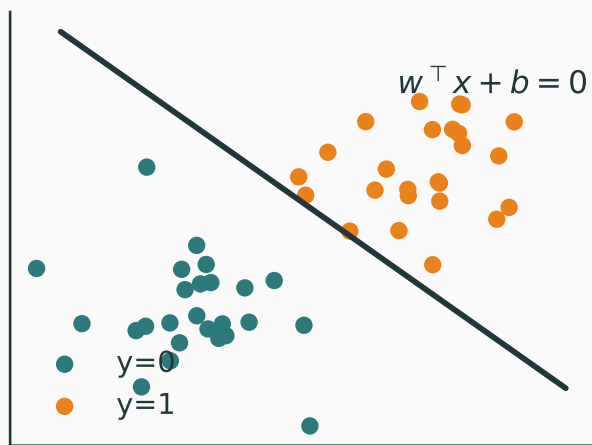
Controls: activation type · input z · show derivative · compare saturation.

Explore large positive and negative logits, where sigmoid's derivative becomes tiny.

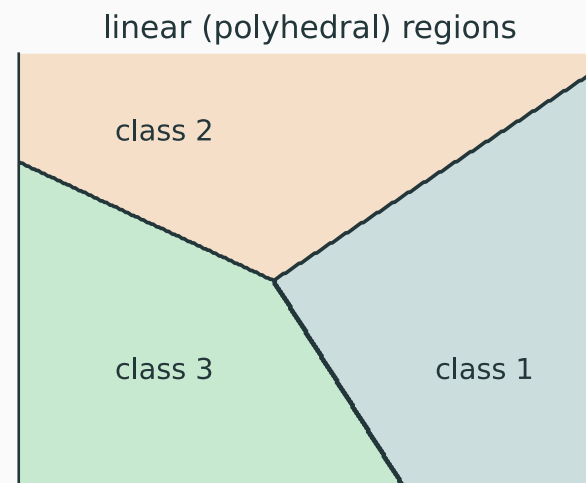
**Where a single weighted sum
fails**

Single-layer classifiers draw only straight boundaries

Binary: threshold one score $z = w^\top x + b$.



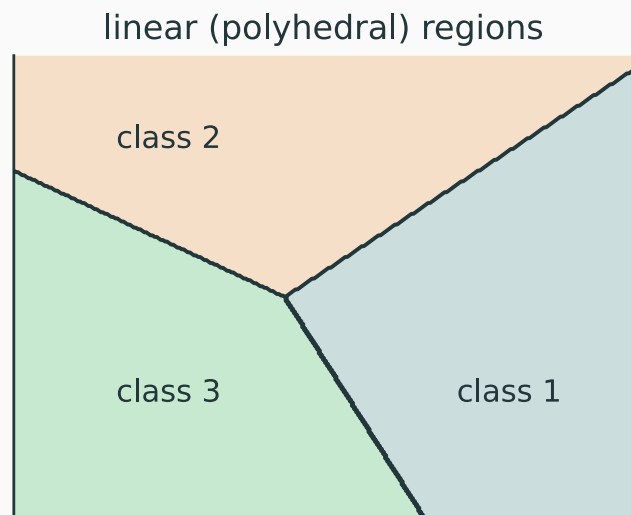
Multi-class: compare several linear scores.



Sigmoid and softmax change scores into probabilities; they do not bend the boundaries in input space.

Question: why are the multi-class boundaries straight?

QUESTION



three linear scores divide the input plane

The classifier predicts

$$\hat{y}(\mathbf{x}) = \arg \max_k p_k(\mathbf{x}).$$

When is a point on the boundary between predicted classes i and j ?

- A.** Whenever $p_i = p_j$
- B.** When $p_i = p_j$ and both are maximal
- C.** Whenever $p_i + p_j = 1$
- D.** Only when all K probabilities are equal

Softmax preserves score comparisons.

$$\frac{p_i}{p_j} = \frac{\exp(z_i)}{\exp(z_j)} = \exp(z_i - z_j).$$

Therefore

$$p_i = p_j \leftrightarrow z_i = z_j,$$

$$p_i > p_j \leftrightarrow z_i > z_j.$$

$\arg \max_k p_k = \arg \max_k z_k$: softmax changes the scale, not the ranking.

Linear-score ties are flat.

$$z_{k(x)} = \mathbf{w}_k^\top \mathbf{x} + b_k$$

$$z_i = z_j \leftrightarrow (\mathbf{w}_i - \mathbf{w}_j)^\top \mathbf{x} + (b_i - b_j) = 0.$$

This equation is a line in 2-D and a hyperplane in higher dimensions.

It is the **candidate pairwise boundary** between classes i and j .

**Pairwise equality explains why every candidate boundary is straight.
But is every tie visible?**

A pairwise tie alone is not enough.

Suppose

$$\mathbf{z} = (1, 1, 3).$$

Then

$$p_1 = p_2 < p_3.$$

The prediction is class 3. The point lies on $z_1 = z_2$, but **not** on a visible class-1/class-2 boundary.

Decision region for class i

$$\mathcal{R}_i = \{\mathbf{x} : z_i \geq z_l \text{ for every } l\}.$$

Boundary shared by classes i and j

$$\mathcal{B}_{ij} = \mathcal{R}_i \cap \mathcal{R}_j$$

$$\mathcal{B}_{ij} = \{\mathbf{x} : z_i = z_j \geq z_l \text{ for every } l\}.$$

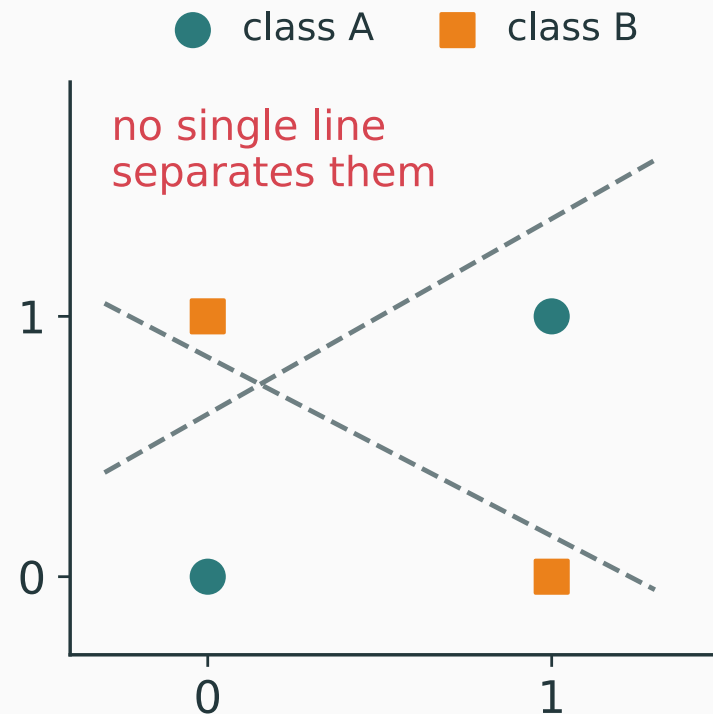
Equivalently, $p_i = p_j$ and both probabilities are maximal.

Correct: B. The visible boundary is the top-scoring portion of a pairwise linear tie set.

XOR exposes the missing ingredient

No single line separates the two XOR classes.

The model first needs a transformation of the input—new features in which the classes become separable.



Hidden nonlinear units learn the transformation instead of asking us to hand-design it.

The multilayer perceptron

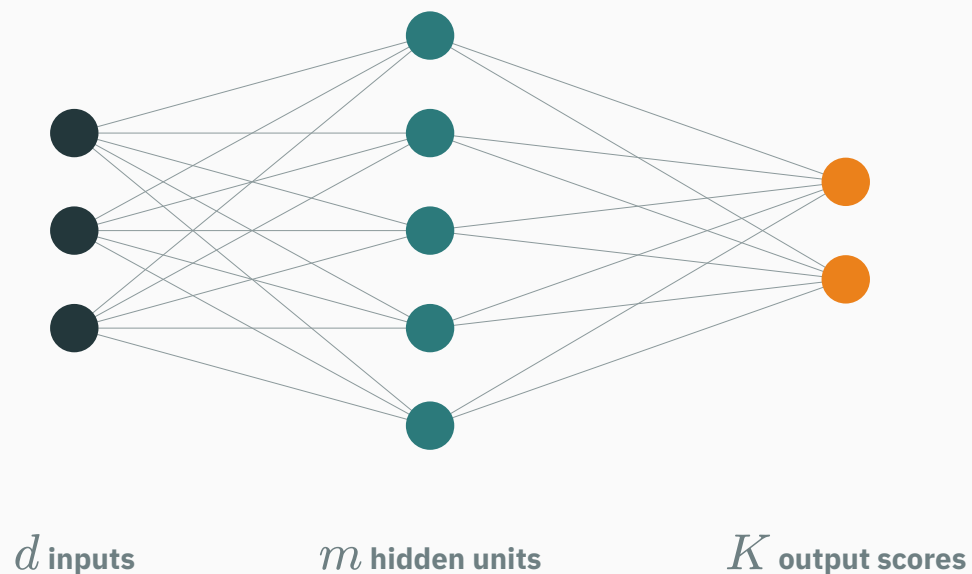
A hidden layer learns features before the output

Instead of $z = \mathbf{W}x + \mathbf{b}$, insert a nonlinear layer first:

$$\mathbf{h} = \varphi(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \mathbf{z} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$

For classification, $\mathbf{p} = \text{softmax}(\mathbf{z})$. This is a **one-hidden-layer MLP**.

Layer dimensions must agree



$$x \in \mathbb{R}^d$$

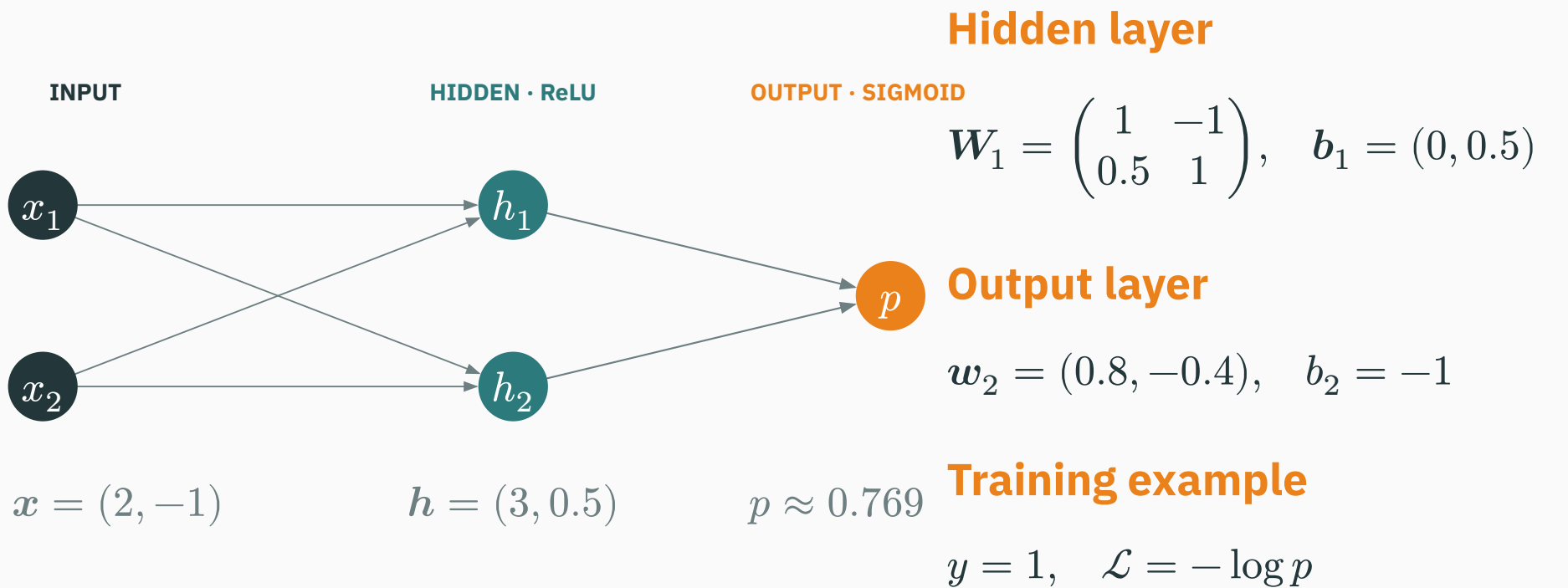
$$W_1 \in \mathbb{R}^{m \times d}, \quad b_1 \in \mathbb{R}^m$$

$$h \in \mathbb{R}^m$$

$$W_2 \in \mathbb{R}^{K \times m}, \quad b_2 \in \mathbb{R}^K$$

$$z \in \mathbb{R}^K$$

First see the complete $2 \rightarrow 2 \rightarrow 1$ network



$$x \rightarrow u = W_1 x + b_1 \rightarrow h = \text{ReLU}(u) \rightarrow z = w_2^\top h + b_2 \rightarrow p = \sigma(z) \rightarrow \mathcal{L}$$

Now calculate the same network from left to right.

Work a $2 \rightarrow 2$ hidden layer by hand

Let $x = (2, -1)$ and

$$\mathbf{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = (0, 0.5).$$

The hidden pre-activation is

$$\mathbf{u} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 = (3, 0.5).$$

Apply ReLU elementwise:

$$\mathbf{h} = \text{ReLU}(\mathbf{u}) = (3, 0.5).$$

The hidden layer converts the raw input into two learned features.

The output layer turns hidden features into a prediction

For binary classification, choose $w_2 = (0.8, -0.4)$ and $b_2 = -1$.

$$z = w_2^\top h + b_2 = 0.8(3) - 0.4(0.5) - 1 = 1.2.$$

$$p = \sigma(1.2) \approx 0.769.$$

If $y = 1$, then $\mathcal{L} = -\log(0.769) \approx 0.262$.

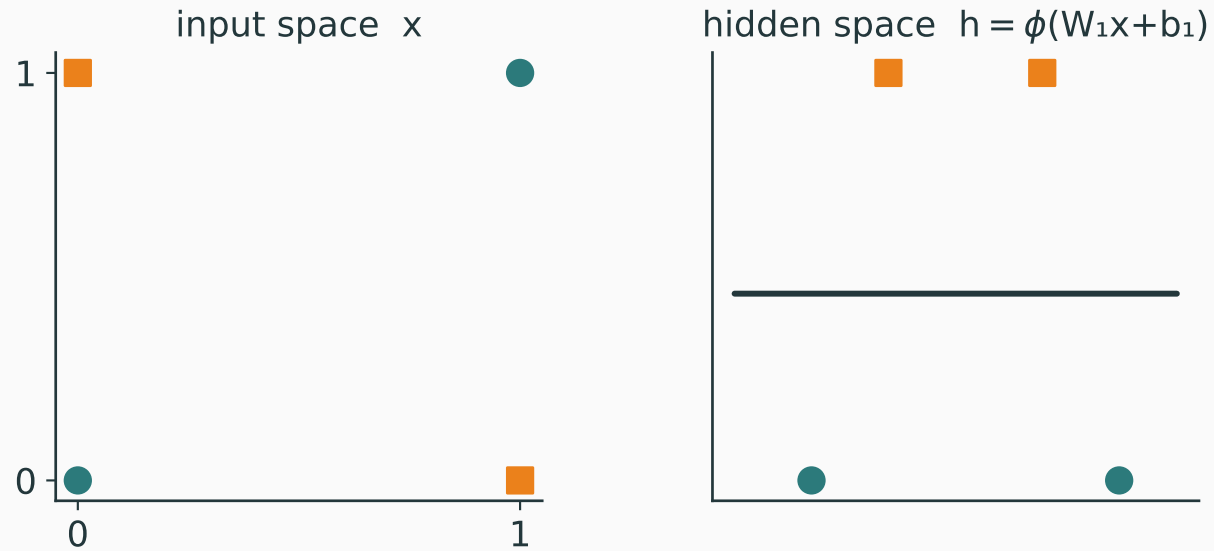
$x \rightarrow h \rightarrow z \rightarrow p \rightarrow \mathcal{L}$ is one complete forward pass.

Each hidden neuron creates one learned feature

$$h_j = \varphi(\mathbf{w}_j^\top \mathbf{x} + b_j)$$

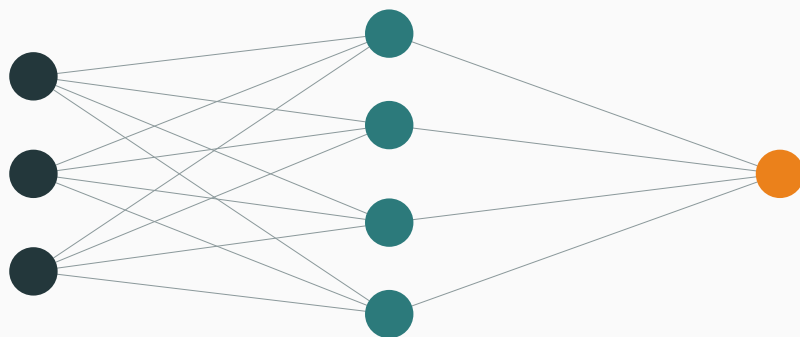
- the first layer **learns features**;
- the output layer **combines** them.

For ReLU, $h_j = \max(0, \mathbf{w}_j^\top \mathbf{x} + b_j)$ — each unit is active on one side of a hyperplane.



An MLP can learn a representation where classification becomes linear.

BINARY CLASSIFICATION



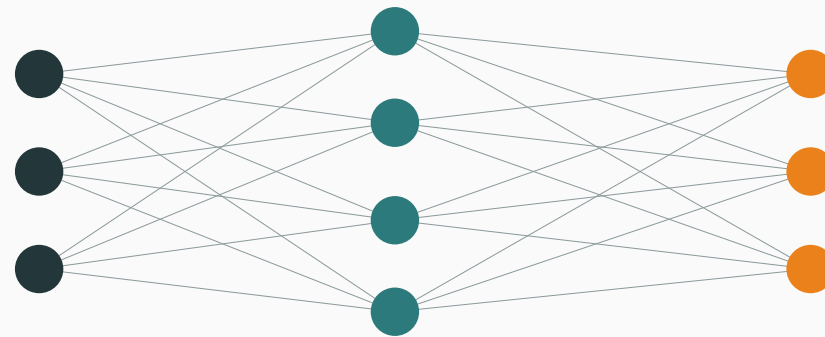
d inputs

m hidden units

1 output node

$$z \in \mathbb{R} \rightarrow p = \sigma(z)$$

MULTI-CLASS CLASSIFICATION



d inputs

m hidden units

K output nodes

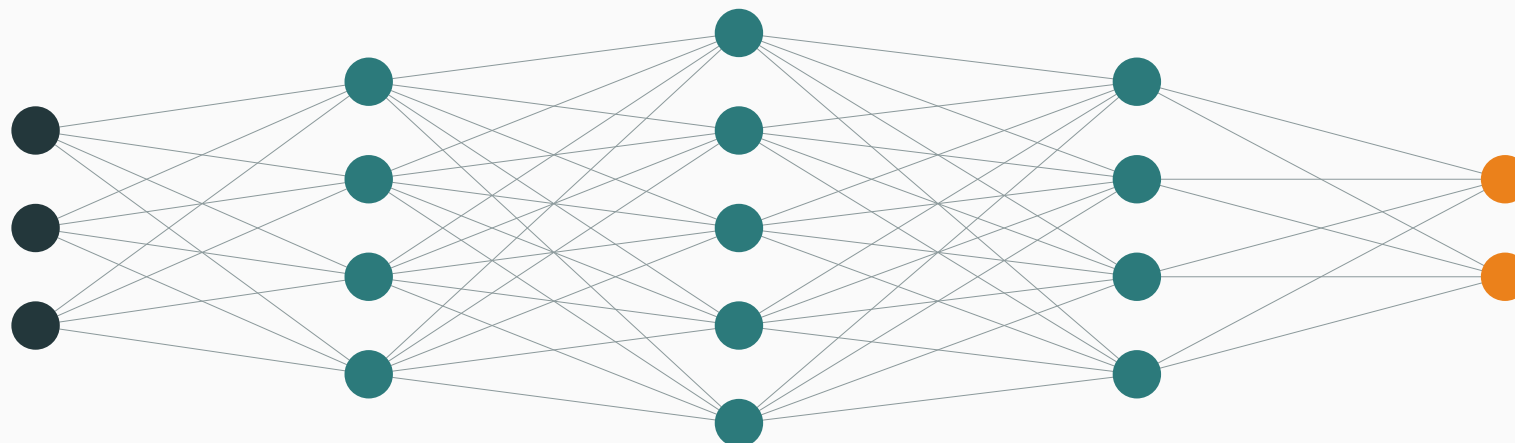
$$z \in \mathbb{R}^K \rightarrow p = \text{softmax}(z)$$

Binary: 1 node gives p and $1 - p$. Multi-class: K nodes give K softmax probabilities.

$$h^{(1)} = \varphi(W_1 x + b_1)$$

$$h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell), \quad \ell = 2, \dots, L-1$$

$$z = W_L h^{(L-1)} + b_L$$

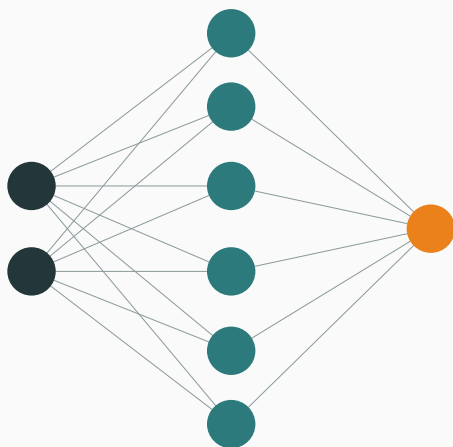


Each hidden layer transforms the previous representation. The output layer then produces task-specific scores or predictions.

Width and depth enlarge a network differently

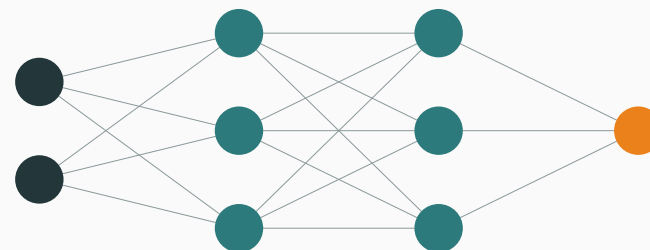
WIDER

One hidden layer, **six neurons**



DEEPER

Two hidden layers, **three neurons each**



Width adds neurons within a layer. **Depth** adds successive transformations.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/mlp-decision-boundary

train a small MLP on **linear / XOR / circles / spirals** and watch the boundary form. Controls: hidden units · layers · activation · learning rate · steps.

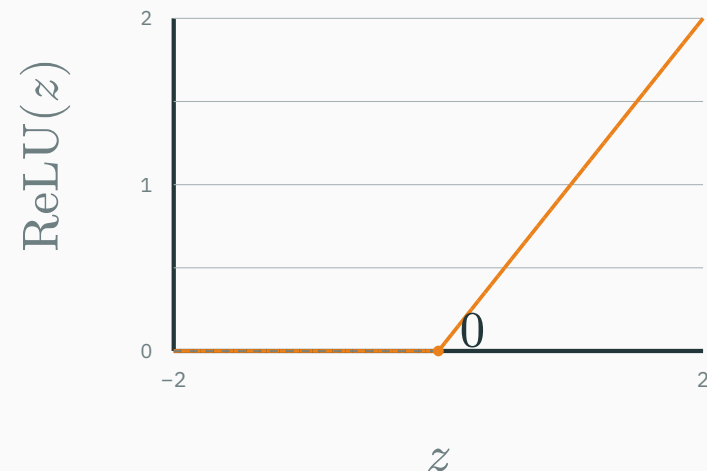
Compare wider layers (more features at one stage) with deeper layers (more composed stages).

Why MLPs represent complex functions

A ReLU contributes one hinge

$$\varphi(z) = \max(0, z)$$

- $z < 0$: output is zero;
- $z > 0$: output grows linearly;
- at $z = 0$: the slope changes—a **hinge**.



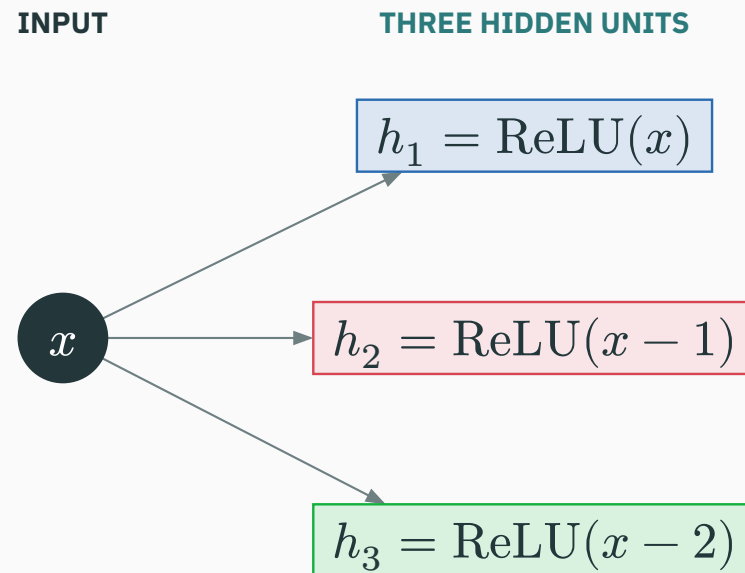
A hidden layer combines many shifted hinges into a piecewise-linear function.

Step 1: one hidden layer creates three shifted ReLUs

Unit 1 uses weight 1 and bias 0: $u_1 = 1x + 0$, $h_1 = \text{ReLU}(u_1) = \text{ReLU}(x)$.

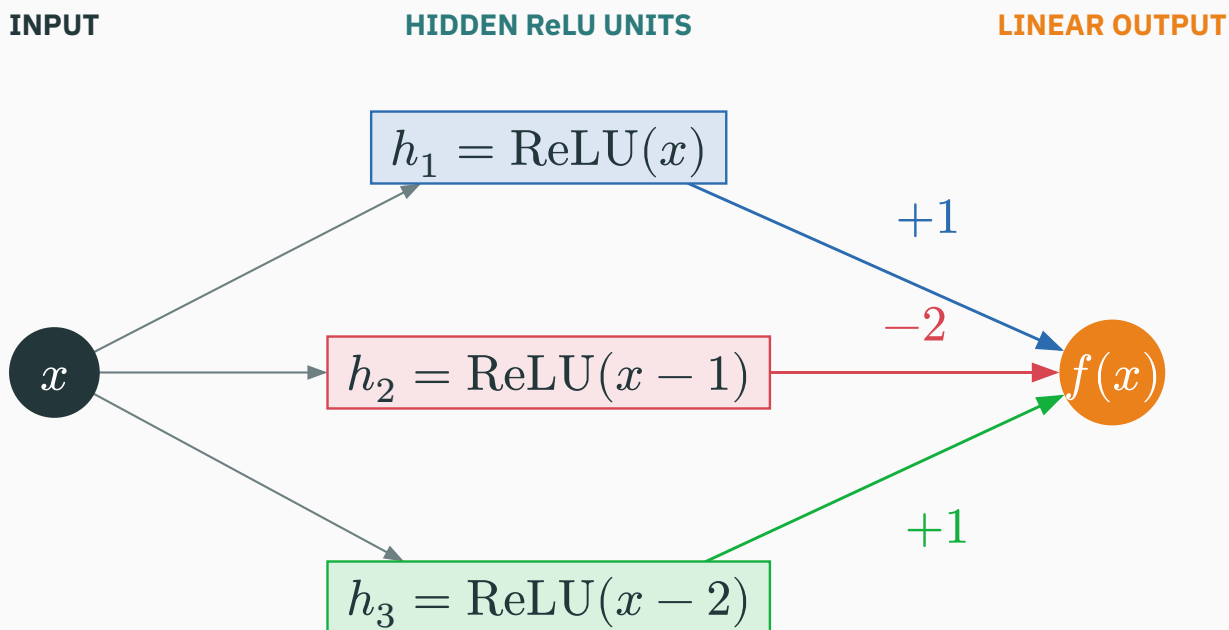
Unit 2 uses weight 1 and bias -1 : $u_2 = 1x - 1$, $h_2 = \text{ReLU}(u_2) = \text{ReLU}(x - 1)$.

Unit 3 uses weight 1 and bias -2 : $u_3 = 1x - 2$, $h_3 = \text{ReLU}(u_3) = \text{ReLU}(x - 2)$.



The hidden biases move the three hinge locations to $x = 0, 1, 2$. No factor -2 has appeared yet.

Step 2: the output layer supplies the coefficients

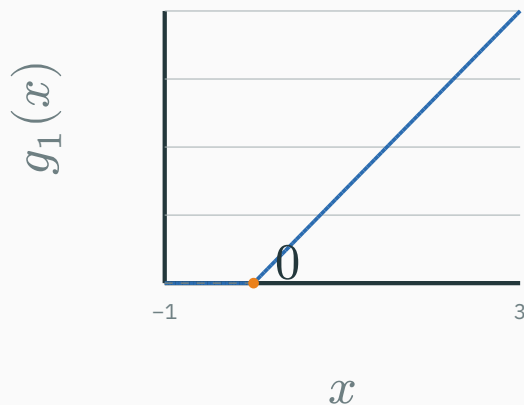


$$h = \text{ReLU} \left(\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} x + \begin{pmatrix} 0 \\ -1 \\ -2 \end{pmatrix} \right), \quad f(x) = (1 \quad -2 \quad 1)h.$$

The -2 is an **output-layer weight**: the second hidden activation is multiplied by -2 only after ReLU.

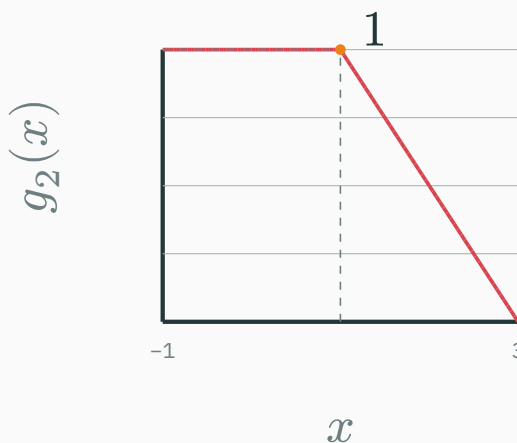
Step 3: inspect the output contributions separately

$$g_1(x) = (+1)h_1$$



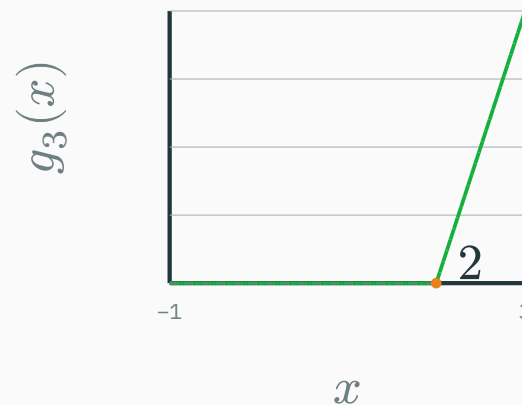
hinge at $x = 0$

$$g_2(x) = (-2)h_2$$



$h_2 \geq 0$; output weight -2 flips it

$$g_3(x) = (+1)h_3$$



hinge at $x = 2$

Hidden activations h_j are the features; $g_j = v_j h_j$ are their signed contributions to the output.

Step 4: add the active terms on each interval

$$f(x) = h_1 - 2h_2 + h_3 = \text{ReLU}(x) - 2\text{ReLU}(x - 1) + \text{ReLU}(x - 2).$$

Input region	h_1	$-2h_2$	h_3	Sum $f(x)$
$x < 0$	0	0	0	0
$0 \leq x < 1$	x	0	0	x
$1 \leq x < 2$	x	$-2(x - 1)$	0	$2 - x$
$x \geq 2$	x	$-2(x - 1)$	$x - 2$	0

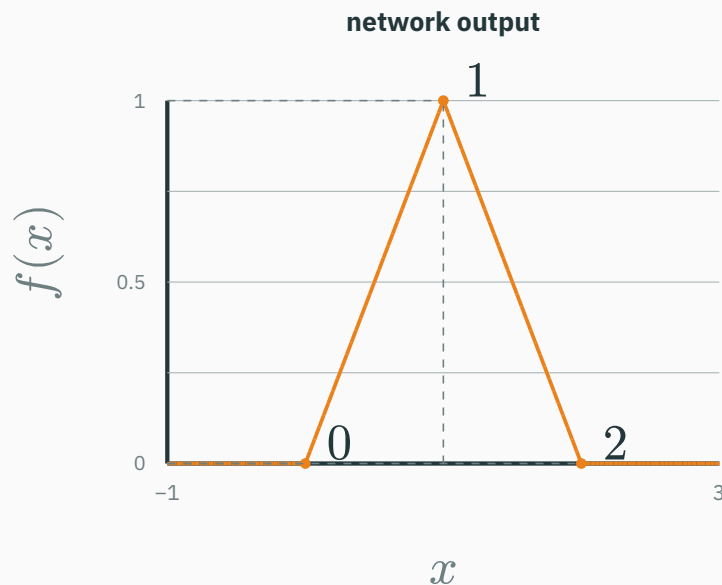
The output rises as x , falls as $2 - x$, then returns to zero: a tent.

Step 5: the weighted sum is the tent

Hidden layer $h(x) =$
 $(\text{ReLU}(x), \text{ReLU}(x - 1), \text{ReLU}(x - 2)).$

Linear output layer $f(x) =$
 $(1 \ -2 \ 1)h(x).$

The network has **three hidden nodes** and **one output node**. The output node performs no ReLU here; it only takes a weighted sum.



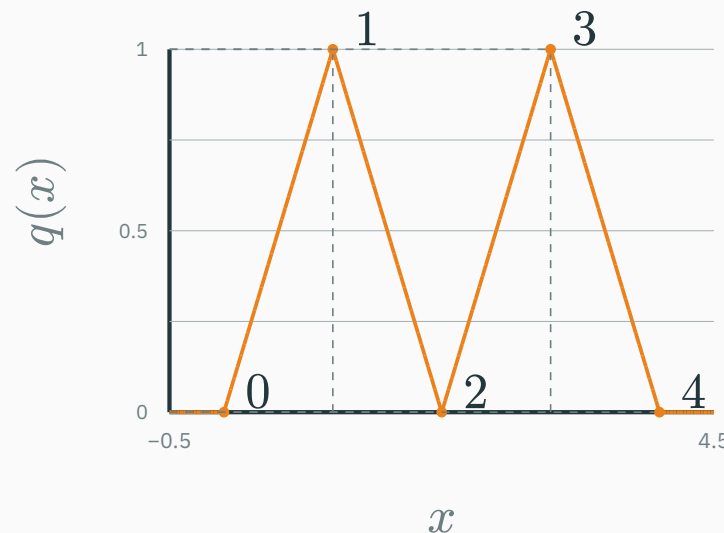
Hidden units create reusable nonlinear features; the output layer chooses their signs and strengths.

Five hinges create a two-peak function

A slightly richer weighted sum:

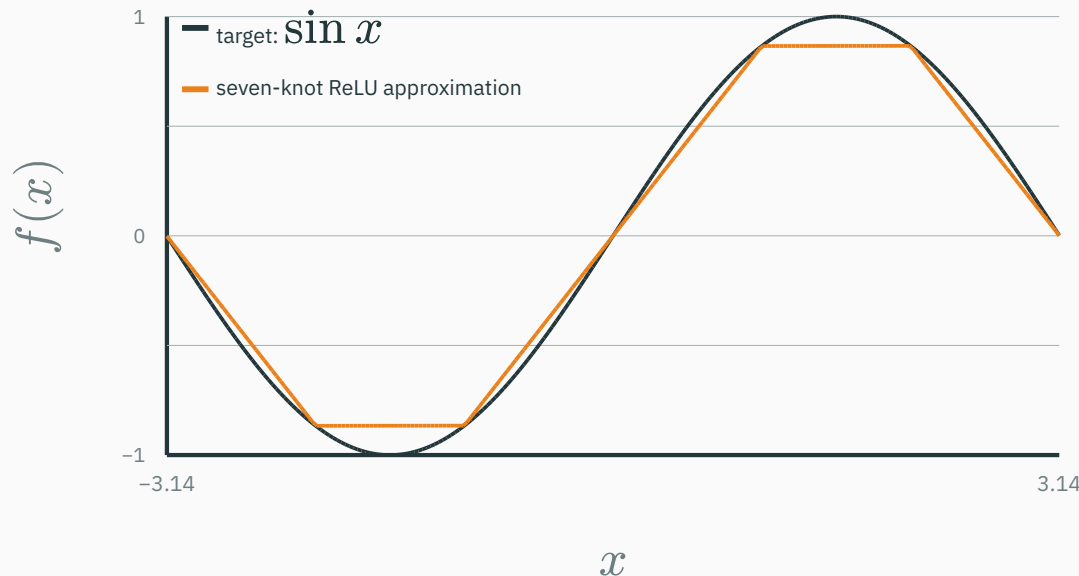
$$\begin{aligned} q(x) = & \text{ReLU}(x) - 2\text{ReLU}(x - 1) \\ & + 2\text{ReLU}(x - 2) - 2\text{ReLU}(x - 3) \\ & + \text{ReLU}(x - 4). \end{aligned}$$

Each shifted unit changes the slope at one breakpoint.



More hidden units $\rightarrow$ more breakpoints $\rightarrow$ a more detailed piecewise-linear function.

More breakpoints can follow a smooth curve



The orange approximation is still made only of straight segments; its corners are the hinges.

Adding hidden units adds breakpoints and reduces the approximation gap.

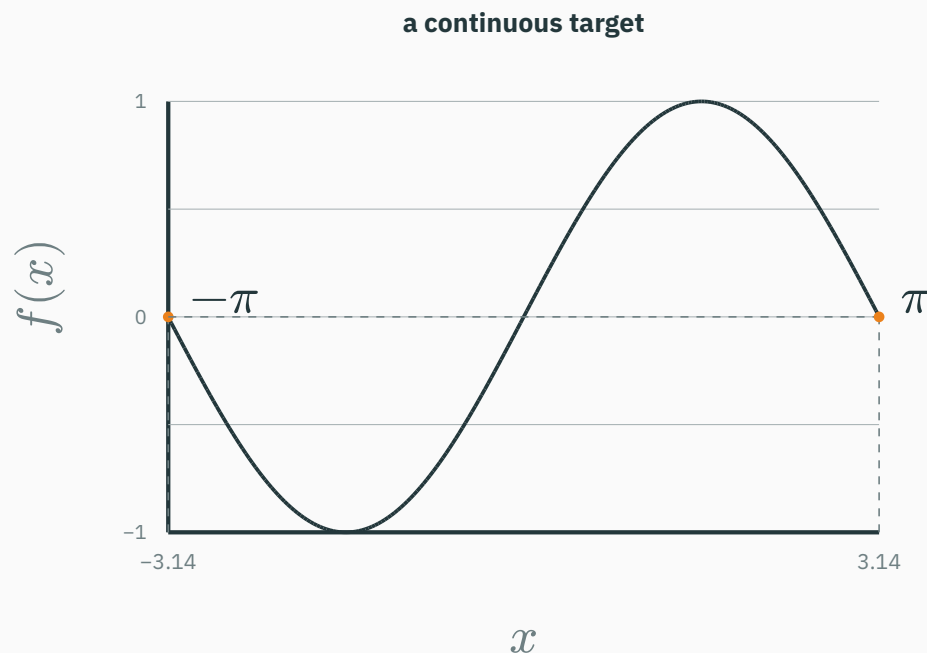
INTERACTIVE

↗ nipunbatra.github.io/dl-teaching/interactives/relu-function-builder.html

Recreate the sequence: inspect one hinge, combine three into a tent, add two more for a two-peak function, then increase the number of knots used to approximate $\sin x$. Controls: preset · hidden-unit count · hinge location · output weight · show components.

Predict which local slope will change **before** moving a hinge or its output weight.

First fix the target and the input region



Continuous means that the target has no jumps.

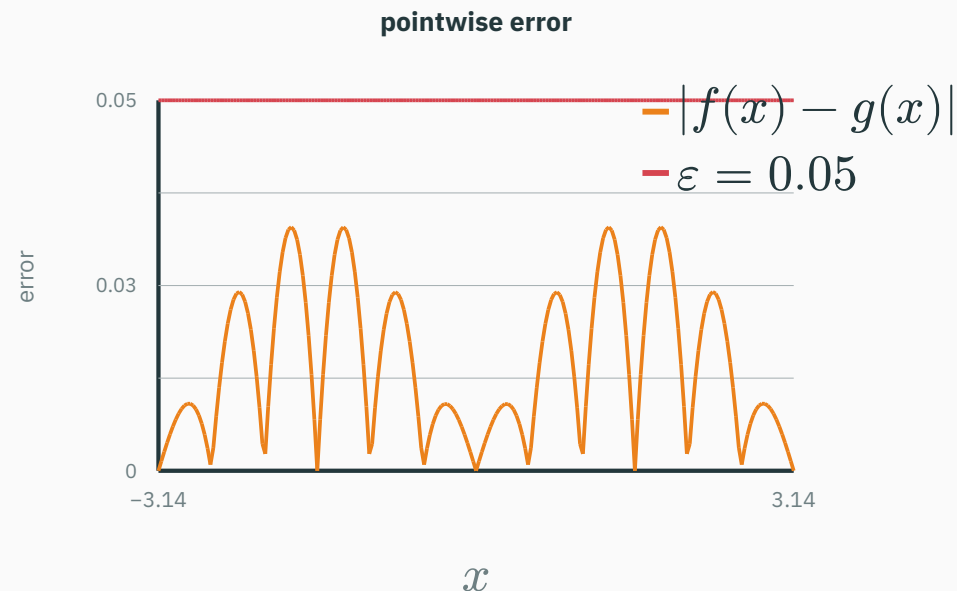
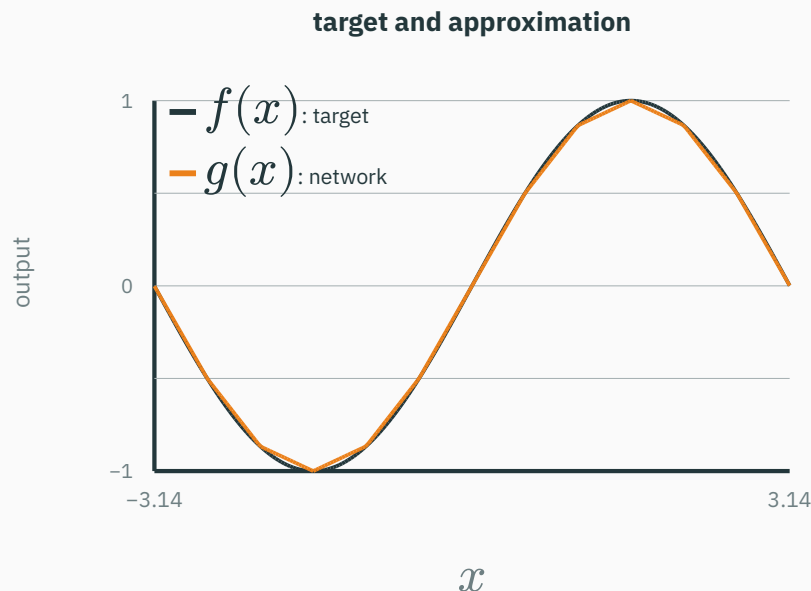
In one dimension, a **compact domain** can be a closed, bounded interval such as

$$D = [-\pi, \pi].$$

We ask the network to match f **on** D ; the theorem makes no claim outside that chosen region.

Approximation begins with one target function on one specified input region.

“Within ε ” means close at every input

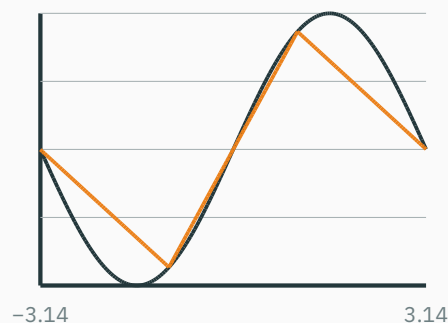


$\sup_{x \in D} |f(x) - g(x)|$ is the **largest vertical gap** anywhere on D .

The guarantee is uniform: no input in the interval may exceed the chosen tolerance.

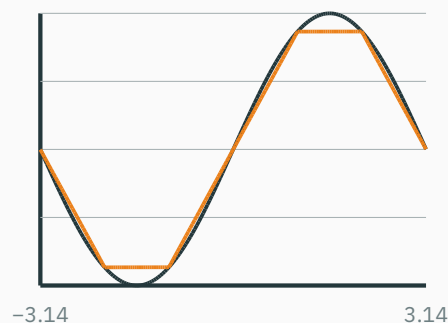
More hinges reduce the largest approximation error

4 knots



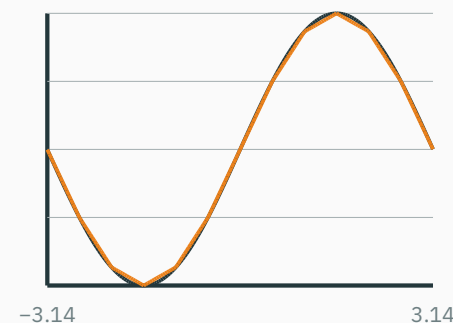
largest error ≈ 0.44

7 knots



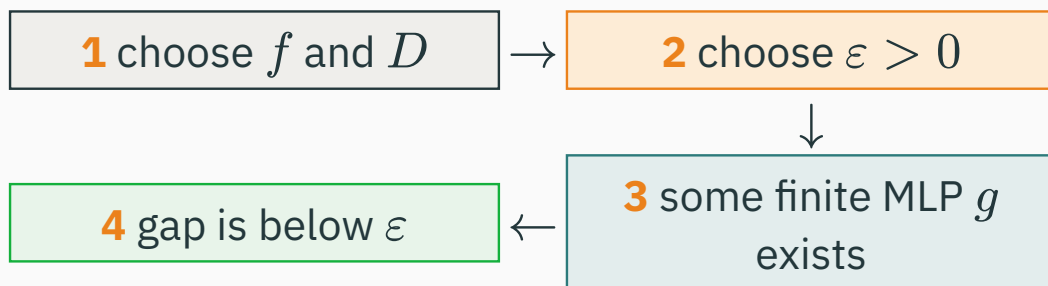
largest error ≈ 0.13

13 knots



largest error ≈ 0.033

More hidden units provide more breakpoints, so a piecewise-linear network can follow finer detail.



The order matters.

For every ε : we may demand any positive tolerance.

There exists g : the network can change when we demand a smaller error.

$$\forall \varepsilon > 0, \exists \text{ MLP } g$$

$$\text{such that } \sup_{x \in D} |f(x) - g(x)| < \varepsilon.$$

This is a statement about representational capacity—not yet about how to find the network.

Universal approximation does **not** promise:

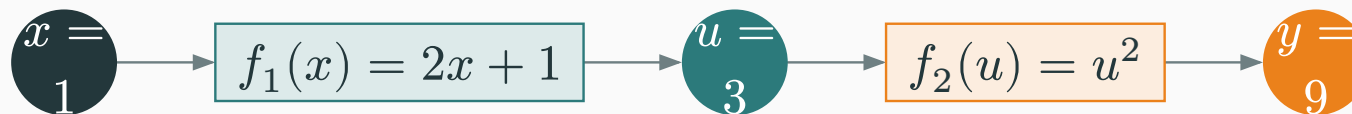
- that training will **find** that function;
- that **finite data** is enough;
- that a **small** network suffices;
- that it will **generalize**.

expressivity $\neq$ trainability $\neq$ generalization

Composition means feeding one result into the next

Start with two small functions:

$$f_1(x) = 2x + 1, \quad f_2(u) = u^2.$$



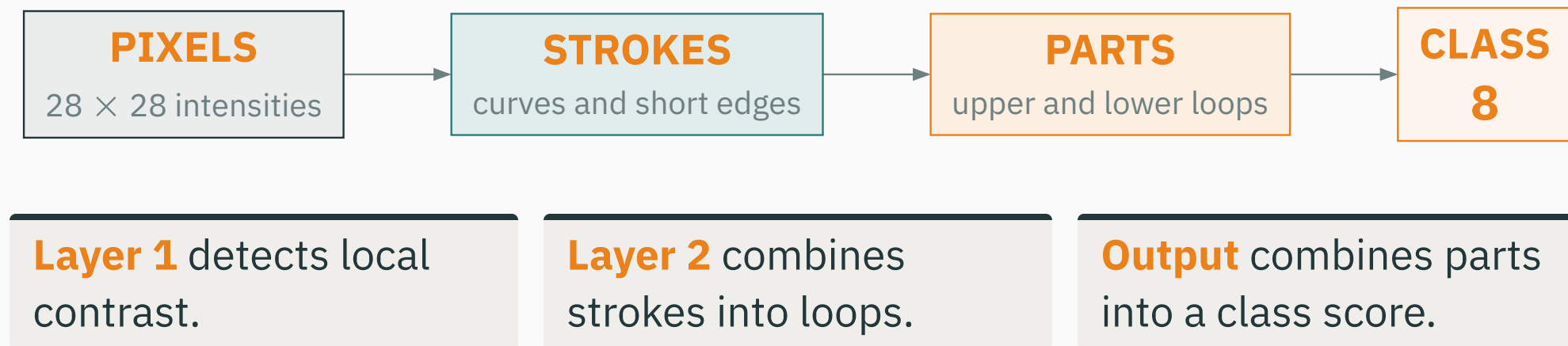
$$y = f_2(f_1(x)) = (2x + 1)^2$$

The value u is an **intermediate feature**: the second stage uses what the first stage computed.

Composition is sequential reuse: $f = f_2 \circ f_1$.

A deep model can build features in stages

Consider recognizing a handwritten **8** from an image:

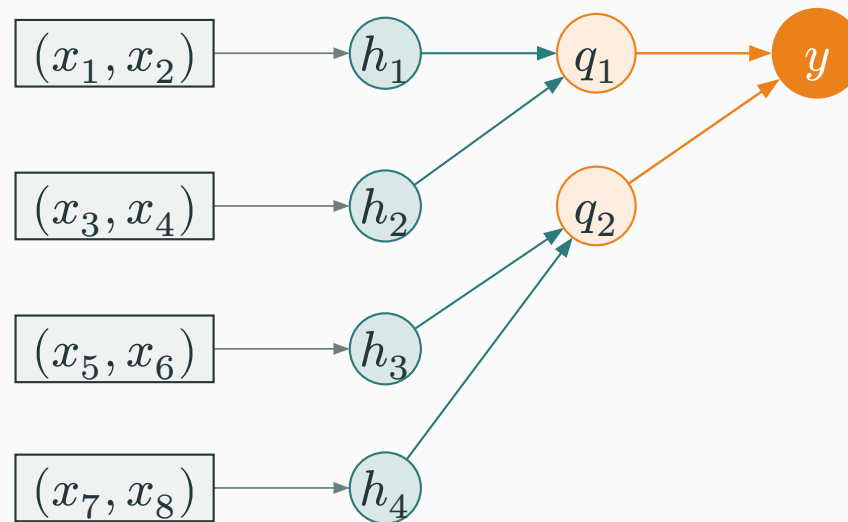


Later features are functions of earlier features: depth mirrors a hierarchy.

Depth can match a compositional problem

Suppose the target itself combines eight inputs in stages:

INPUTS $\rightarrow$ PAIR FEATURES $\rightarrow$ GROUP FEATURES $\rightarrow$ OUTPUT

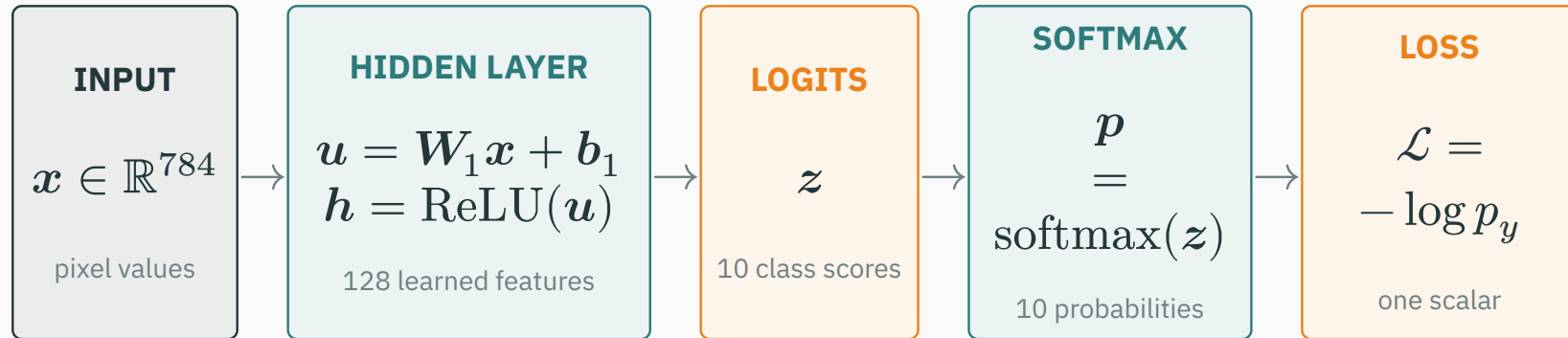


Each node combines two values, yet y depends on all eight. For some compositional functions, a shallow network needs far more units.

Connecting to deep-learning practice

A forward pass transforms one input, step by step

Example: classify one 28×28 image — 784 pixel values.

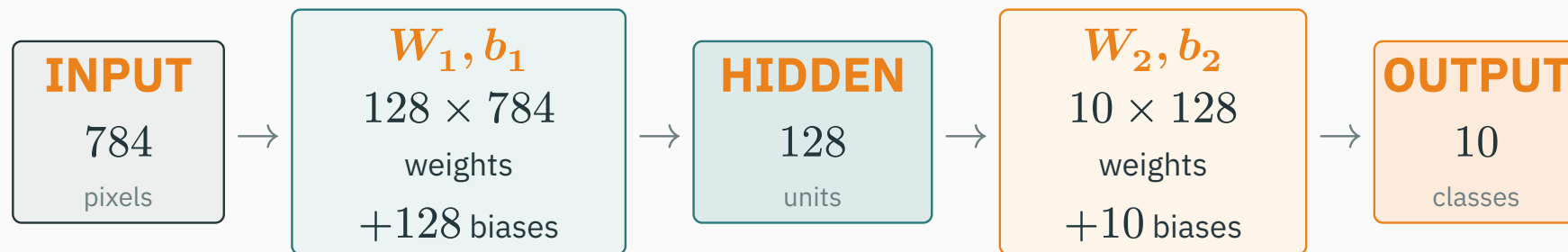


The true class y selects its predicted probability p_y ; a smaller p_y gives a larger loss.

A forward pass computes a prediction and its loss; it does **not update the weights.**

Count parameters by following the same network

Every connection has a **weight**; every receiving unit has a **bias**.



Input $\rightarrow$ hidden

weights: $128 \times 784 = 100,352$

biases: 128

subtotal: 100,480

Hidden $\rightarrow$ output

weights: $10 \times 128 = 1,280$

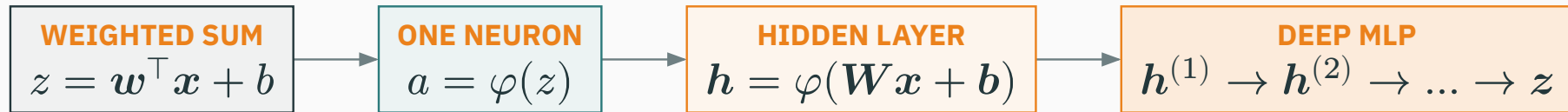
biases: 10

subtotal: 1,290

Layer ℓ : $d_\ell d_{\ell-1}$ weights + d_ℓ biases.

total = $100,480 + 1,290 = 101,770$ **trainable parameters**

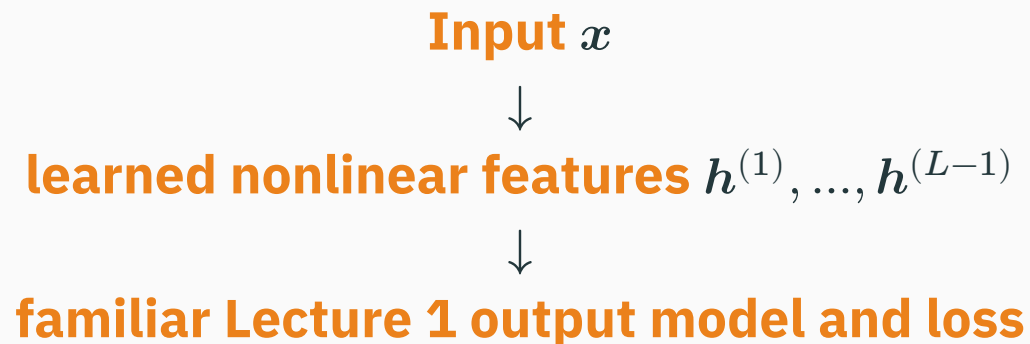
From one weighted sum to an MLP



Stage	Representation	What it adds
Weighted sum	$z = \mathbf{w}^\top \mathbf{x} + b$	One line or plane; captures a global linear trend.
One neuron	$a = \varphi(z)$	One nonlinear feature: a hinge, gate, or score.
Hidden layer	$\mathbf{h} = \varphi(\mathbf{W}\mathbf{x} + \mathbf{b})$	Many learned features can bend or partition the input space.
Deep MLP	$\mathbf{h}^{(\ell)} = \varphi(\mathbf{W}_\ell \mathbf{h}^{(\ell-1)} + \mathbf{b}_\ell)$	Layers compose features into a hierarchical representation.

The key addition is not a new loss; it is a learned nonlinear representation.

The decisive change is the learned representation



The output loss can stay the same while hidden layers make the representation nonlinear.

We now have $x \rightarrow f_{\theta}(x) \rightarrow \mathcal{L}$.

Lecture 3: how do we compute $\nabla_{\theta}\mathcal{L}$ efficiently?

computation graphs · the chain rule · backpropagation · automatic differentiation